

B11: Meta-Circular Abstraction II

CS1101S: Programming Methodology

Martin Henz

November 3, 2023

- 1 Review
- 2 A small complication: return statements
- 3 While and for loops
- 4 Arrays

- 1 Review
- 2 A small complication: return statements
- 3 While and for loops
- 4 Arrays

Review

Most fundamental idea in programming

The evaluator, which determines the meaning of statements and expressions in a programming language, is just another program.

- A calculator language: primitive operators, literals, evaluate
- Adding booleans, conditionals, sequences
- Adding blocks and declarations
- Adding simple compound functions

Evaluating the language

```
function evaluate(program) {  
  let C = list(program);  
  let S = null;  
  let E = the_global_environment;  
  while (! is_null(C)) {  
    const command = head(C);  
    C = tail(C);  
    if ... // handle expressions and statements  
        // handle instructions  
  } else {  
    error("unknown command");  
  } }  
  return head(S);  
}
```

Recall simple compound functions

```

stmt ::= ...
        | function name ( params ) {
            return expr ; }           function declaration

expr ::= ...
        | expr( expr1, ..., exprn)   function application
        | params => expr               lambda expression

```

Example:

```

function fact(n) {
    return n === 1 ? 1 : n * fact(n - 1);
}
fact(5); // result: 120

```

Recall evaluating compound functions

```
while (! is_null(C)) {  
    const command = head(C); ...  
    if ... else if (is_lambda_expr(command)) {  
        if (is_return_stmt(lambda_body(command))) {  
            S = pair(make_simple_function(  
                    lambda_parameter_symbols(command),  
                    return_expr(lambda_body(command)),  
                    E), S);  
        } else {  
            error("body must be return statement");  
        }  
    } else if ...  
}
```

Recall function objects

Example: $x \Rightarrow x + y$

```
function make_simple_function(params, body, env) {  
    return list("simple_function",  
                parameters, body, env);  
}
```

Recall evaluation of call instructions

Example: call 1

```
while (! is_null(C)) {  
  const command = head(C); C = tail(C);  
  if ... else if (is_call_instr(command)) {  
    const arity = call_instr_arity(command);  
    let args = null;  
    let n = arity;  
    while (n > 0) {  
      args = pair(head(S), args);  
      S = tail(S);  
      n = n - 1; }  
    const fun = head(S); S = tail(S);  
    ...  
  }
```

Recall evaluation of call instructions

Example: call 1

```
...
const fun = head(S); S = tail(S);
if (is_primitive_function(fun)) {
    S = pair(apply_prim_function(fun, args), S);
} else if (is_simple_function(fun)) {
    C = pair(function_body(fun),
             pair(make_env_instr(E), C));
    E = extend_environment(function_params(fun),
                          args, function_environment(fun));
} else { error("unknown function type"); }
} else if ...
} }
```

- 1 Review
- 2 A small complication: return statements**
- 3 While and for loops
- 4 Arrays

Return statements in JavaScript

The Problems

- Functions that do not evaluate a return statement must return undefined
- Evaluation of a return statement *anywhere* in the function body will return from the function with the result of evaluating the return expression

Adding return statements

$$\begin{array}{lcl} stmt & ::= & \dots \\ & | & \text{return } expr ; \text{ return statement} \end{array}$$

Example:

```
function f(x, error) {  
  if (error) {  
    return "error has occurred";  
  }  
  const result = something_complicated(x);  
  return result;  
}  
f(42, true);
```

Recall evaluation of call instructions

Example: call 1

```
...
if (is_primitive_function(fun)) {
    S = pair(apply_prim_function(fun, args), S);
} else if (is_simple_function(fun)) {
    C = pair(function_body(fun),
             pair(make_env_instr(E), C));
    E = extend_environment(function_params(fun),
                          args, function_environment(fun));
} else { error("unknown function type"); }
} else if ...
} }
```

Adding a marker

Example: call 1

```
...  
if (is_primitive_function(fun)) { ...  
} else if (is_simple_function(fun)) { ...  
} else { // handle complex function call  
    C = pair(function_body(fun),  
             pair(make_marker(),  
                  pair(make_env_instruction(E),  
                      C)));  
    E = extend_environment(  
        function_parameters(fun),  
        args,  
        function_environment(fun));  
} ...
```

Handling return statements

Example: `return something_complicated(x);`

```
while (! is_null(C)) {  
    const command = head(C); ...  
    if ... else if (is_return_statement(command)) {  
        C = pair(return_expression(command),  
                  pair(make_return_instruction(),  
                        C));  
    } else if ...  
} }
```

Handling return instructions

Example: `return`

```
while (! is_null(C)) {  
    const command = head(C); ...  
    if ... else if (is_return_instr(command)) {  
        if (is_marker(head(C))) {  
            C = tail(C);  
        } else {  
            C = pair(command, tail(C));  
        }  
    } else if ...  
} }
```

Recall evaluating compound functions

```
while (! is_null(C)) {  
  const command = head(C); ...  
  if ... else if (is_lambda_expr(command)) {  
    if (is_return_stmt(lambda_body(command))) {  
      S = pair(make_simple_function(  
                lambda_parameter_symbols(command),  
                return_expr(lambda_body(command)),  
                E), S);  
    } else {  
      error("body must be return statement");  
    }  
  } else if ...  
}
```

Handling missing return

```
if ... else if (is_lambda_expr(command)) {  
    if (is_return_stmt(lambda_body(command))) { ...  
    } else {  
        S = pair(make_complex_function(  
                    lambda_parameter_symbols(command),  
                    make_sequence(  
                        list(lambda_body(command),  
                            make_return_statement(  
                                make_literal(undefined))),  
                    E),  
                S);  
    }  
} else if ...  
} }
```

Complex function objects

Example: $x \Rightarrow \{ \text{if } (\dots) \{ \text{return } \dots \} \text{return } \dots \}$

```
function make_complex_function(params, body, env) {  
    return list("complex_function",  
               parameters, body, env);  
}
```

- 1 Review
- 2 A small complication: return statements
- 3 While and for loops**
- 4 Arrays

Evaluation of while loops

```
while (! is_null(C)) {  
  const command = head(C); ...  
  if ... else if (is_while_loop(command)) {  
    C = pair(  
      make_conditional_statement(  
        while_loop_predicate(command),  
        make_sequence(  
          list(while_loop_body(command),  
              command)),  
          make_sequence(null)),  
      C);  
  } else if ...  
}
```

Evaluation of For-loops: A first approximation

```
for (E1; E2; E3) { statements }
```

is an abbreviation for

```
{ E1;  
  while (E2) {  
    statements  
    E3;  
  }  
}
```

In our evaluator: A first approximation

```
} else if (is_for_loop(command)) {  
  C = pair(  
    make_block(  
      make_sequence(  
        list(  
          for_loop_init(command),  
          make_while_loop(  
            for_loop_predicate(command),  
            make_sequence(  
              list(  
                for_loop_body(command),  
                for_loop_update(command)  
              ))))))) , C);
```

A strange example

```
const a = [];  
for (let i = 0; i < 10; i = i + 1) {  
    a[i] = x => x + i;  
}  
a[5](100);
```

JavaScript design decision

```
const a = [];  
for (let i = 0; i < 10; i = i + 1) {  
    a[i] = x => x + i;  
}  
a[5](100);
```

Special treatment of loop control variable

Each iteration of the body has its own “copy” of the loop control variable.

Therefore, the result of the program is 105.

Evaluation of array expressions

```
while (! is_null(C)) {  
    const command = head(C); ...  
    if ... else if (is_arr_expr(command)) {  
        const arr_exprs = arr_elements(command);  
        C = append(arr_exprs ,  
                    pair(make_arr_instr(  
                        length(arr_exprs)),  
                        C));  
    } else if ...  
} }
```

Evaluation of array instructions

```
while (! is_null(C)) {  
  const command = head(C); ...  
  if ... else if (is_arr_instr(command)) {  
    const arr = [];  
    const size = arr_instr_size(command);  
    for (let i = size - 1; i >= 0; i = i - 1) {  
      arr[i] = head(S);  
      S = tail(S);  
    }  
    S = pair(arr, S);  
  } else if ...  
} }
```

Evaluation of array access

```
while (! is_null(C)) {  
  const command = head(C); ...  
  if ... else if (is_arr_access(command)) {  
    C = pair(arr_access_arr_expr(command),  
            pair(arr_access_index_expr(command),  
                pair(make_arr_access_instr(),  
                    C)));  
  } else if ...  
} }
```

Evaluation of array access instructions

```
while (! is_null(C)) {  
    const command = head(C); ...  
    if ... else if (is_arr_access_instr(command)) {  
        const index = head(S);  
        const arr = head(tail(S));  
        S = pair(arr[index], tail(tail(S)));  
    } else if ...  
} }
```

Evaluation of array assignment

```
while (! is_null(C)) {  
  const command = head(C); ...  
  if ... else if (is_arr_assignment(command)) {  
    const access =  
      arr_assignment_access(command);  
    C = pair(arr_access_arr_expr(access),  
            pair(arr_access_index_expr(access),  
                pair(arr_assignment_value_expr(  
                    command),  
                    pair(make_arr_assignment_instr(),  
                        C))));  
  } else if ...  
}
```

Evaluation of array assignment instructions

```
while (! is_null(C)) {  
  const command = head(C); ...  
  if ... else if (is_arr_asmt_instr(command)) {  
    const val = head(S);  
    const index = head(tail(S));  
    const arr = head(tail(tail(S)));  
    arr[index] = val;  
    S = pair(val, tail(tail(tail(S))));  
  } else if ...  
} }
```

Summary

- Return statements anywhere
- While and for loops
- Arrays
- Challenge: Can you make our evaluator truly *meta-circular*?
Can you get it to evaluate itself?